

Machine Learning for Biology and Health

CSCI 1851
Spring 2026

Ritambhara Singh

February 26, 2026
Thursday

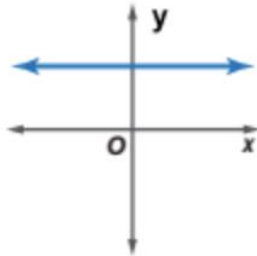


Polynomial regression

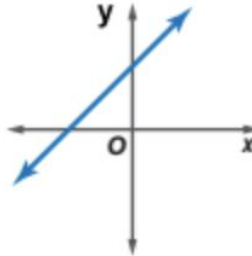
Non-linear function

$$y = \beta_0 + \beta_1 x + \beta_2 x^2 + \cdots + \beta_p x^p + \varepsilon$$

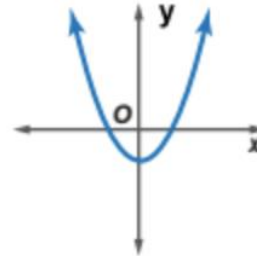
Constant function
Degree 0



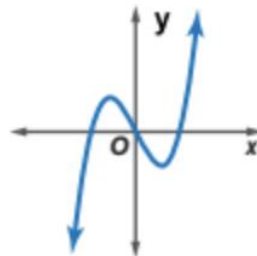
Linear function
Degree 1



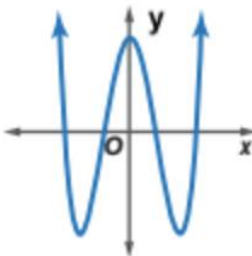
Quadratic function
Degree 2



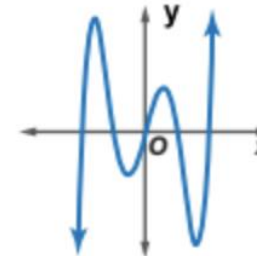
Cubic function
Degree 3



Quartic function
Degree 4



Quintic function
Degree 5



<http://www.math.glencoe.com/>

Today's goal – Learn about kernels and support vector machine (SVM)

- (1) Kernelized Perceptron
- (2) Linear SVM
- (3) Defining maximum margin
- (4) SVM Optimization
- (5) Class activity: SVM

Kernelized Perceptron

Original Perceptron Algorithm

Task: Binary Classification

$$y_i \in \{-1, +1\}$$

Model

$$f(x) = w \cdot x$$

Prediction

$$a = \text{sign}(w \cdot x)$$

Update rule (when model makes mistake)

$$w \leftarrow w + y_i x_i$$

Algorithm 29 PERCEPTRONTRAIN($\mathbf{D}$, $MaxIter$)

```
1:  $w \leftarrow 0, b \leftarrow 0$  // initialize weights and bias
2: for  $iter = 1 \dots MaxIter$  do
3:   for all  $(x, y) \in \mathbf{D}$  do
4:      $a \leftarrow w \cdot \phi(x) + b$  // compute activation for this example
5:     if  $ya \leq 0$  then
6:        $w \leftarrow w + y \phi(x)$  // update weights
7:        $b \leftarrow b + y$  // update bias
8:     end if
9:   end for
10: end for
11: return  $w, b$ 
```

Another way to look at the weight vector

Suppose we make mistakes on points x_1, x_3, x_3, x_7 .

$$w = y_1x_1 + y_3x_3 + y_3x_3 + y_7x_7$$

Grouping the terms

$$w = \sum_i \alpha_i y_i x_i$$

Here: α_i = number of times x_i caused an update

The weight vector is just a weighted combination of training points

Reformulating prediction function

The prediction was:

$$f(x) = w \cdot x$$

Substituting the new formulation

$$f(x) = \left(\sum_i \alpha_i y_i x_i \right) \cdot x$$

Distributing dot product

$$f(x) = \sum_i \alpha_i y_i (x_i \cdot x)$$

Note: The algorithm now uses only dot products of training samples, no need for explicit weight vectors

Kernel Trick

If the algorithm only depends on dot product:

$$x_i \cdot x$$

We can replace that with a KERNEL

$$K(x_i, x)$$

Where:

$$K(x_i, x) = \phi(x_i) \cdot \phi(x)$$

Without ever computing $\phi(x)$.

So the new prediction is:

$$f(x) = \sum_i \alpha_i y_i K(x_i, x)$$

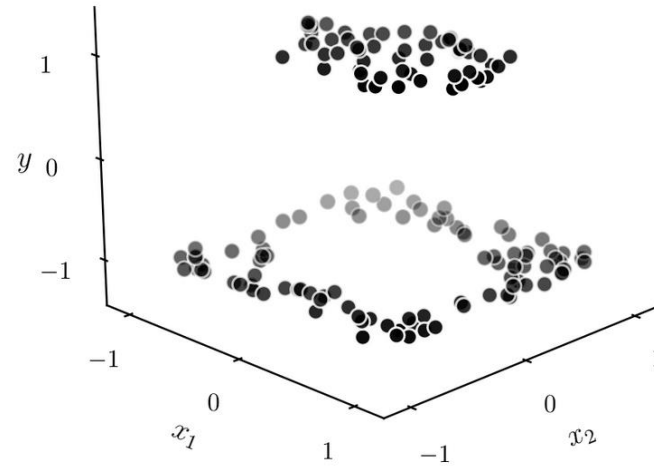
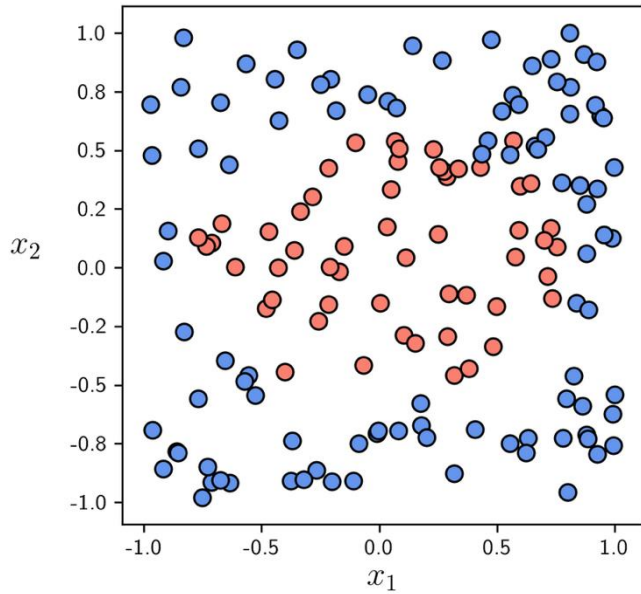
We have designed our Kernelized Perceptron

Algorithm 29 PERCEPTRONTRAIN($\mathbf{D}$, $MaxIter$)

```
1:  $w \leftarrow 0, b \leftarrow 0$  // initialize weights and bias
2: for  $iter = 1 \dots MaxIter$  do
3:   for all  $(x, y) \in \mathbf{D}$  do
4:      $a \leftarrow w \cdot \phi(x) + b$  // compute activation for this example
5:     if  $y a \leq 0$  then
6:        $w \leftarrow w + y \phi(x)$  // update weights
7:        $b \leftarrow b + y$  // update bias
8:     end if
9:   end for
10: end for
11: return  $w, b$ 
```

Algorithm 30 KERNELIZEDPERCEPTRONTRAIN($\mathbf{D}$, $MaxIter$)

```
1:  $\alpha \leftarrow 0, b \leftarrow 0$  // initialize coefficients and bias
2: for  $iter = 1 \dots MaxIter$  do
3:   for all  $(x_n, y_n) \in \mathbf{D}$  do
4:      $a \leftarrow \sum_m \alpha_m \phi(x_m) \cdot \phi(x_n) + b$  // compute activation for this example
5:     if  $y_n a \leq 0$  then
6:        $\alpha_n \leftarrow \alpha_n + y_n$  // update coefficients
7:        $b \leftarrow b + y$  // update bias
8:     end if
9:   end for
10: end for
11: return  $\alpha, b$ 
```



Any questions?



Algorithm 30 KERNELIZEDPERCEPTRONTRAIN($\mathbf{D}$, $MaxIter$)

```

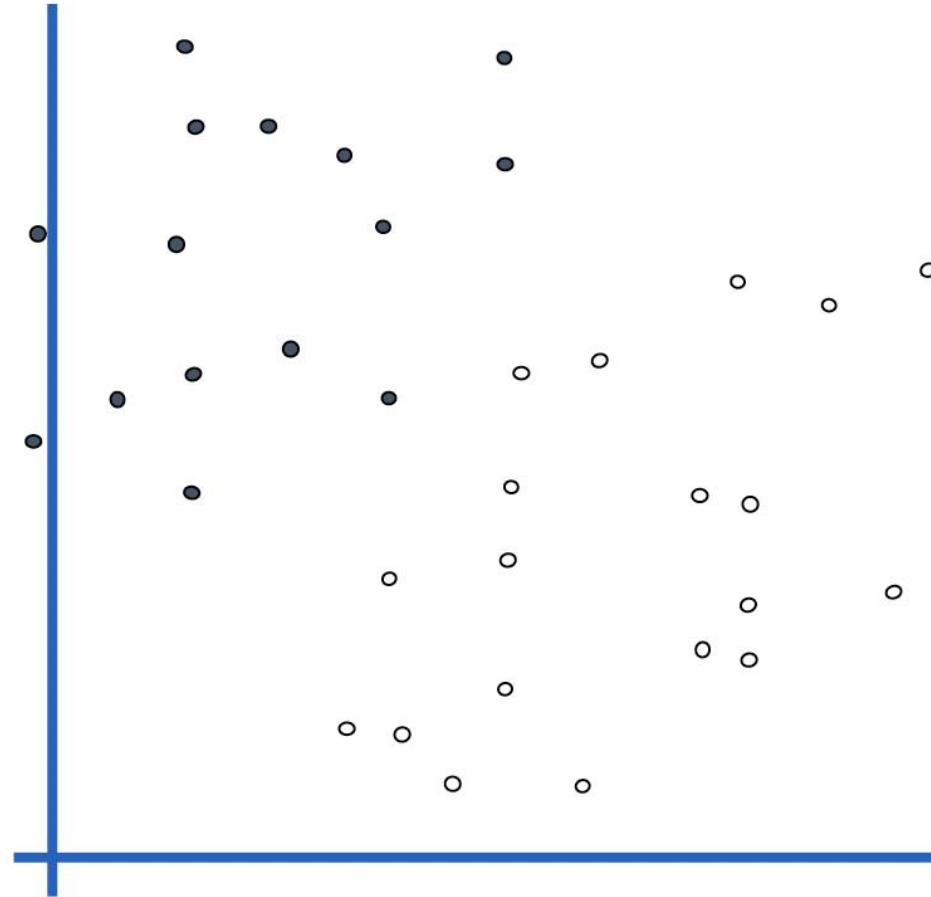
1:  $\alpha \leftarrow 0, b \leftarrow 0$  // initialize coefficients and bias
2: for  $iter = 1 \dots MaxIter$  do
3:   for all  $(x_n, y_n) \in \mathbf{D}$  do
4:      $a \leftarrow \sum_m \alpha_m \phi(x_m) \cdot \phi(x_n) + b$  // compute activation for this example
5:     if  $y_n a \leq 0$  then
6:        $\alpha_n \leftarrow \alpha_n + y_n$  // update coefficients
7:        $b \leftarrow b + y$  // update bias
8:     end if
9:   end for
10: end for
11: return  $\alpha, b$ 

```

Linear Support Vector Machine (SVM)

We are back to Linear Classification..

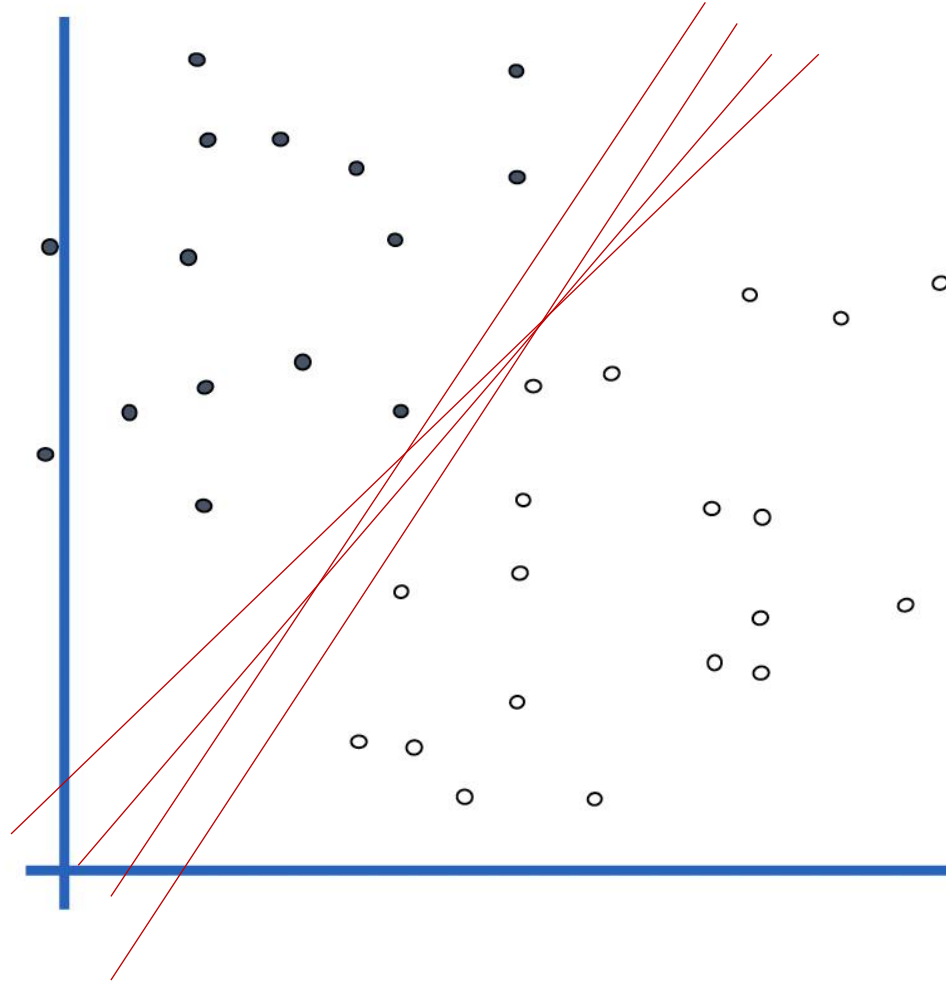
- denotes +1
- denotes -1



How would you
classify this data?

We are back to Linear Classification..

- denotes +1
- denotes -1



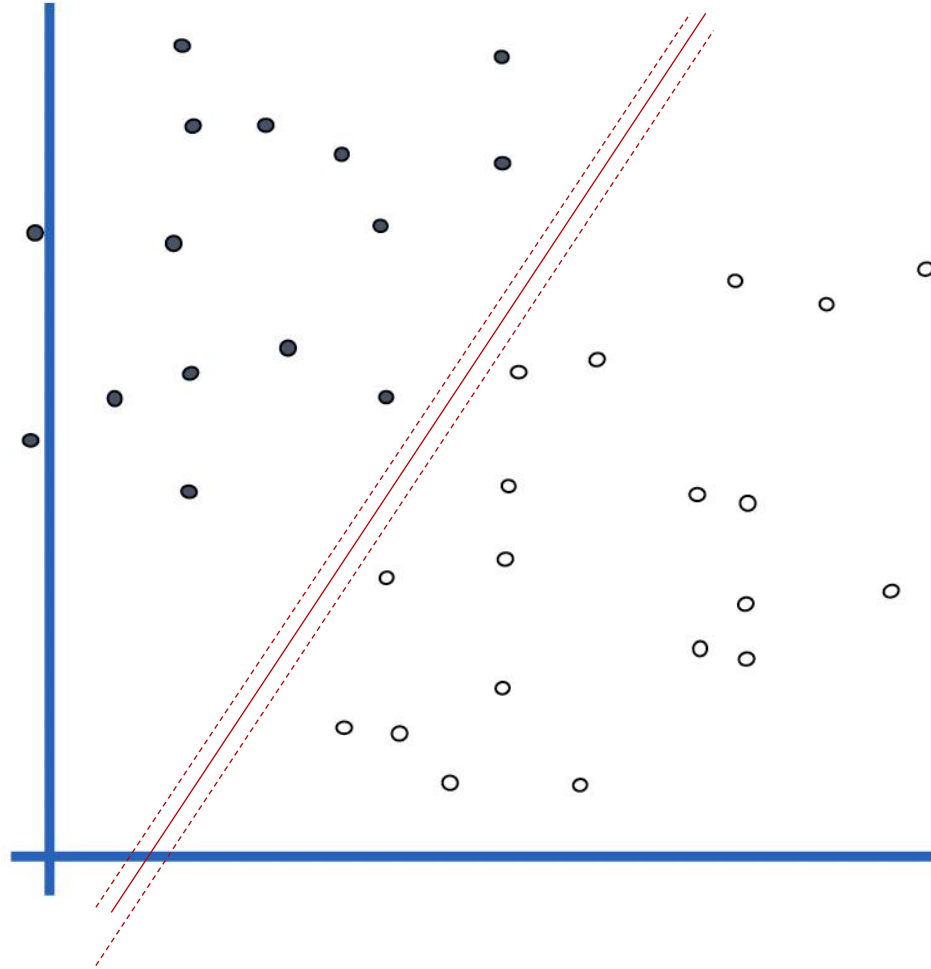
How would you
classify this data?

Any of these would
be fine..

..but which is best?

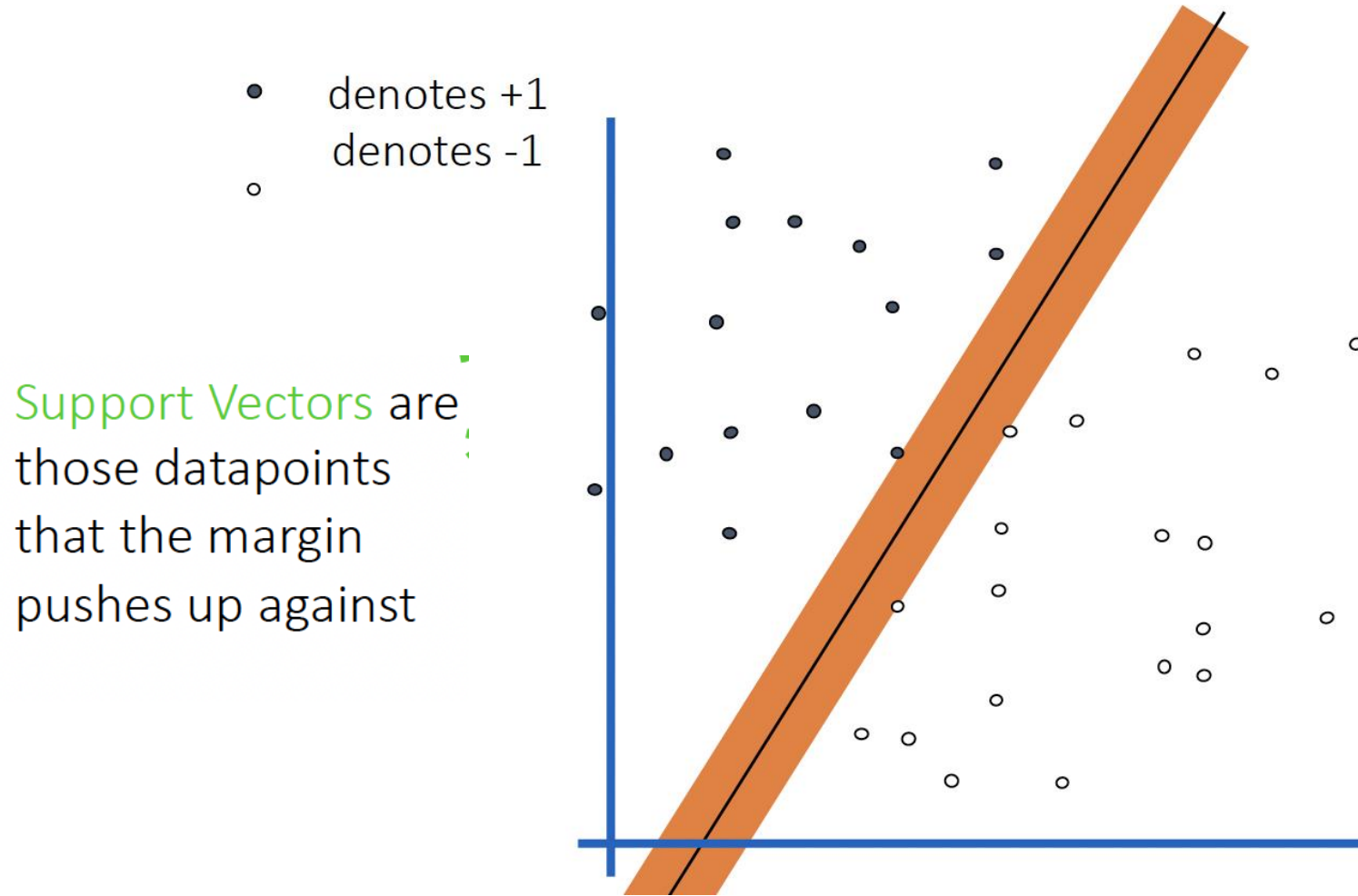
Classifier margin

- denotes +1
- denotes -1



Define the **margin** of a linear classifier as the width that the **boundary could be increased by** before hitting a datapoint.

Maximum Margin



The **maximum margin linear classifier** is the linear classifier with the, **maximum** margin. This is the simplest kind of SVM (Called an LSVM)

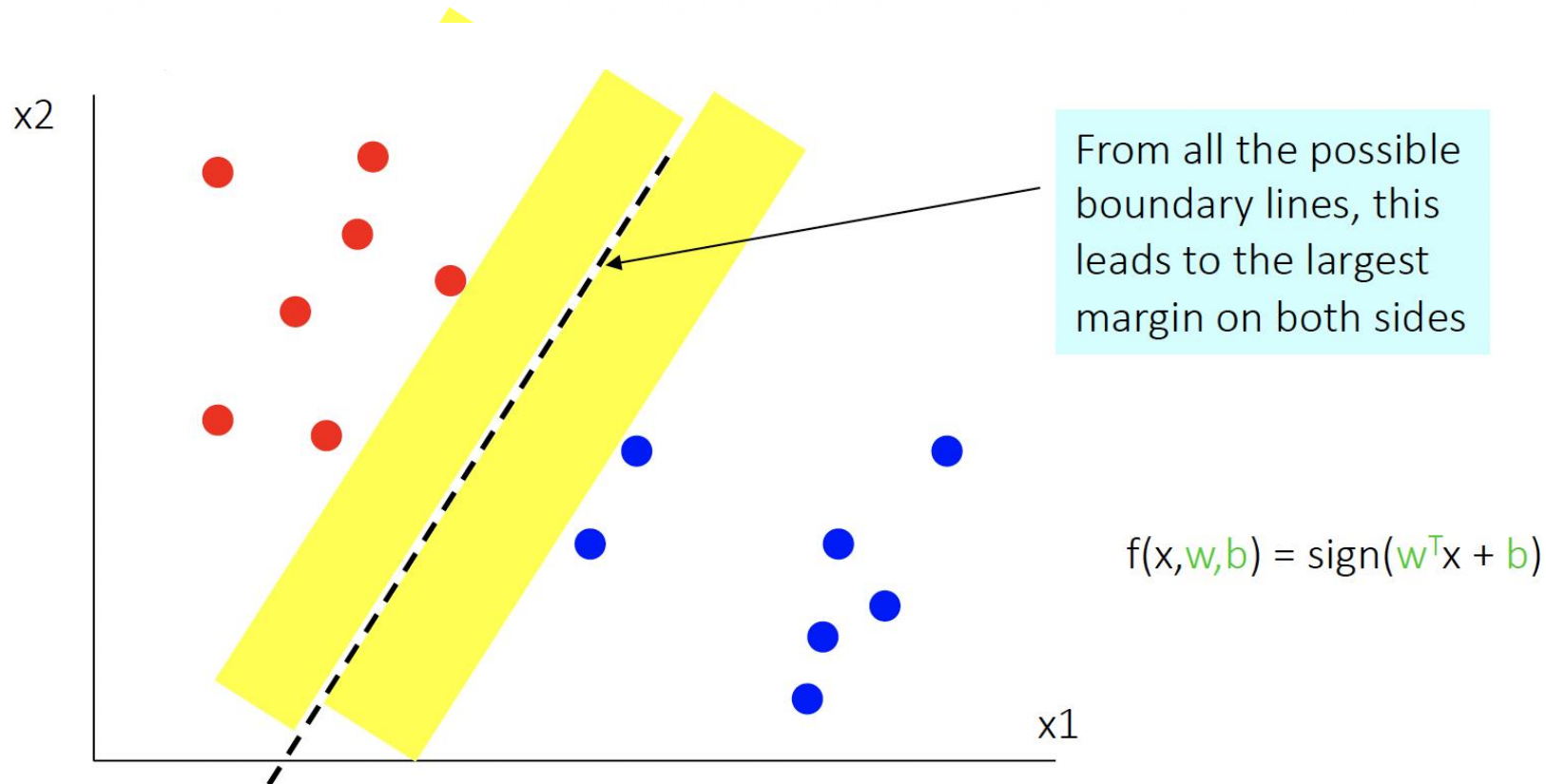
Linear SVM

Maximum Margin Classifier

- Instead of fitting all points, focus on boundary points
- Learn a boundary that leads to the largest margin from both sets of points

Why MAX margin?

- Intuitive, 'makes sense'
- Some theoretical support (using VC dimension)
- Works well in practice



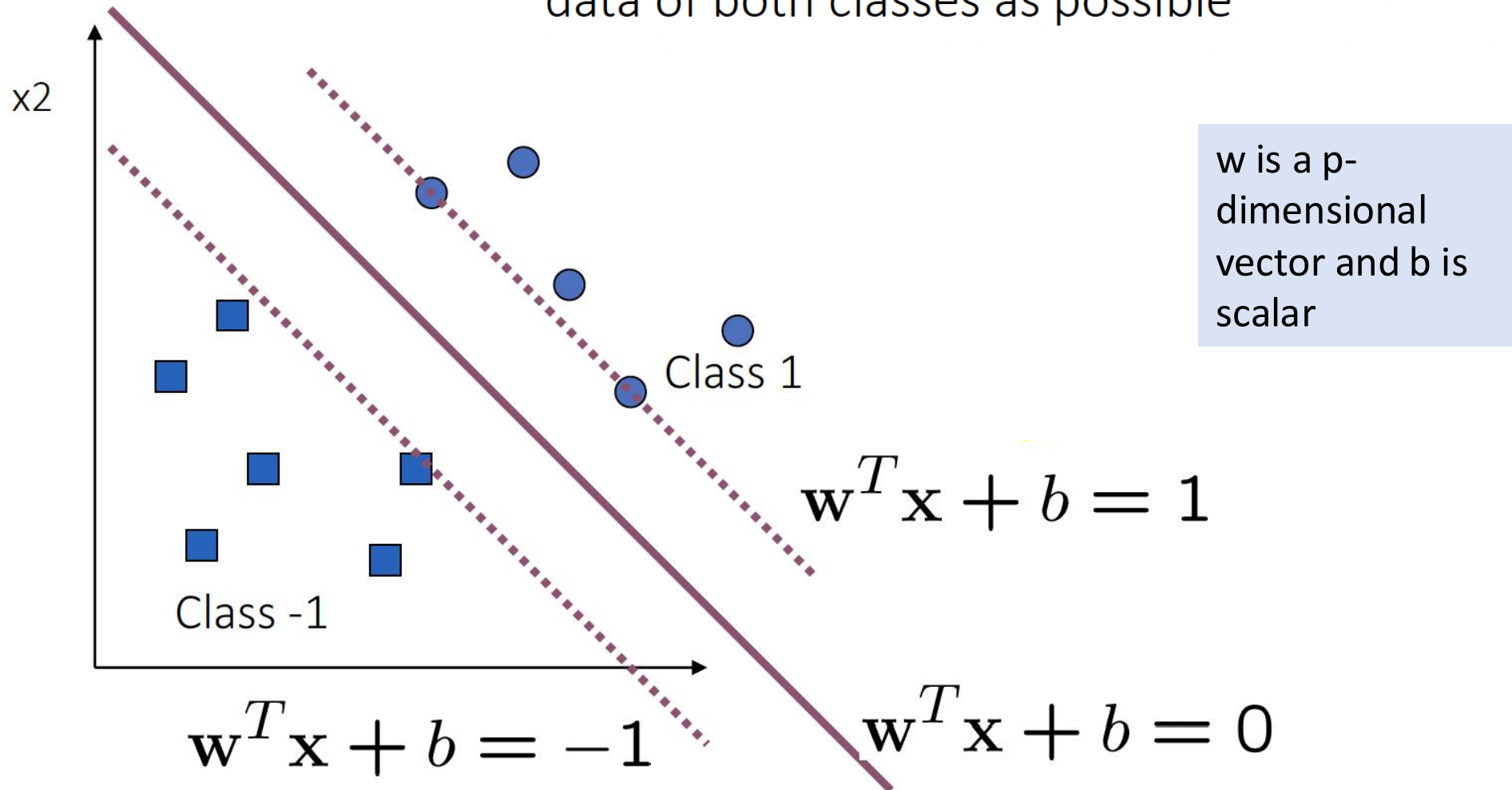
Any questions?



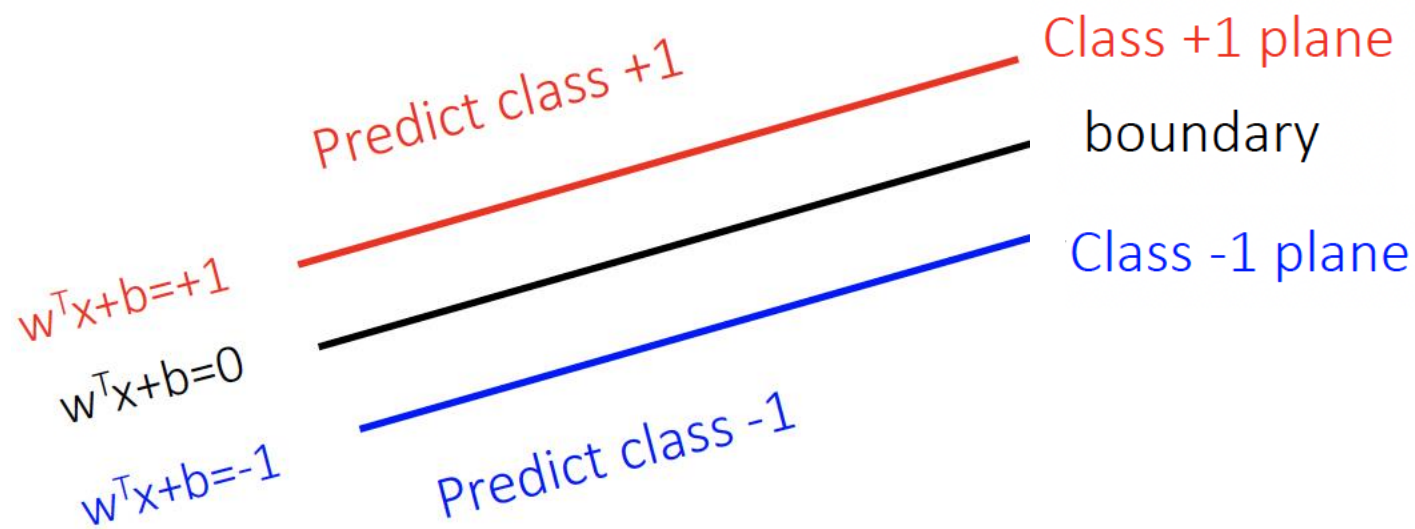
Defining the maximum margin

Maximum Margin and Decision Boundary

- The decision boundary should be as far away from the data of both classes as possible



Specifying a maximum margin classifier



Is the linear separation assumption realistic?

We will deal with this shortly, but let's assume it for now

Classify as +1 if $w^T x + b \geq 1$

Classify as -1 if $w^T x + b \leq -1$

Undefined if $-1 < w^T x + b < 1$

Review:

For two vectors x and y ,

$$x^T y$$

is called the (inner) vector product.

x and y are called orthogonal if

$$x^T y = 0$$

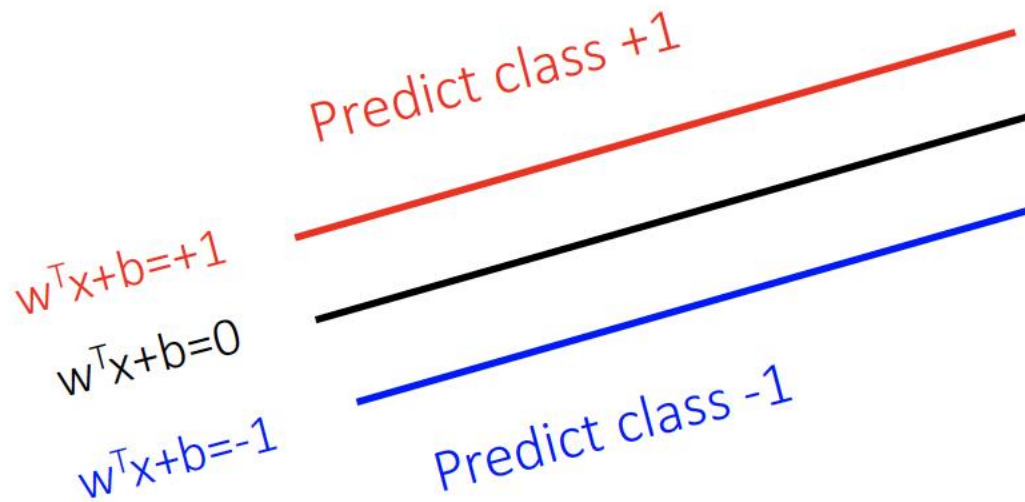
The square root of the product of a vector with itself,

$$\sqrt{x^T x}$$

is called the 2-norm ($|x|_2$), can also write as $|x|$

Euclidean length
of the vector.

Defining the width of the margin



Classify as +1 if $w^T x + b \geq 1$

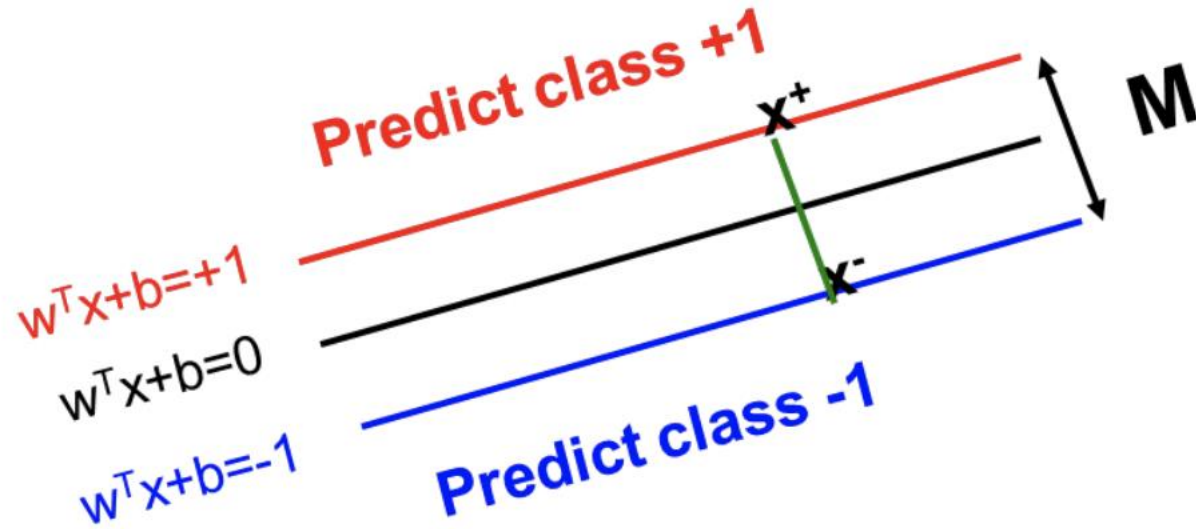
Classify as -1 if $w^T x + b \leq -1$

Undefined if $-1 < w^T x + b < 1$

What do you think we need to calculate?

- Let's define the width of the margin by M
- How can we encode our goal of maximizing M in terms of our parameters (w and b)?

Let M = width of the margin

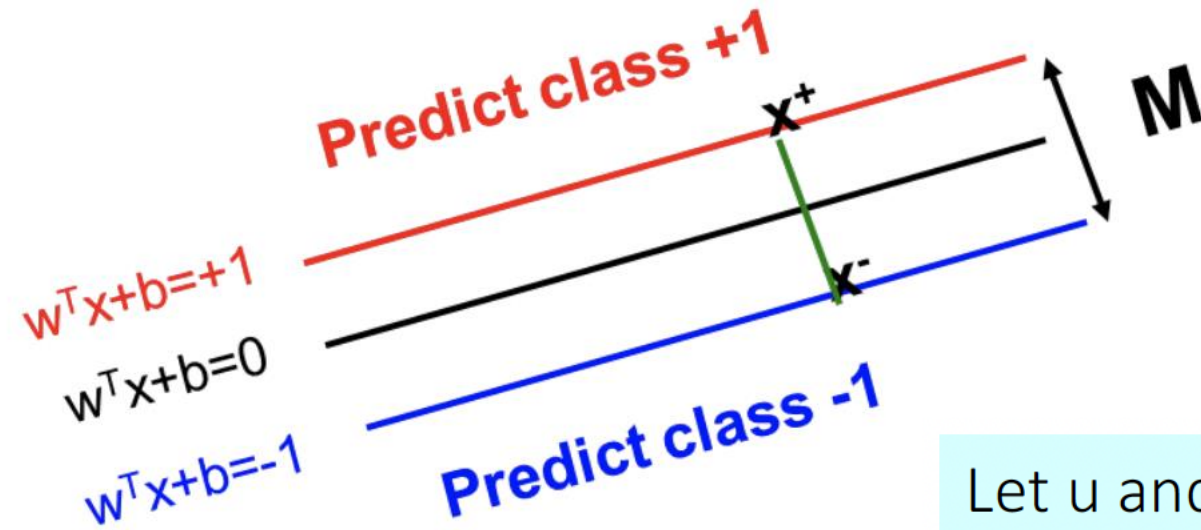


Classify as +1 if $w^T x + b \geq 1$

Classify as -1 if $w^T x + b \leq -1$

Undefined if $-1 < w^T x + b < 1$

Calculating M: Observation 1



Classify as +1 if $w^T x + b \geq 1$

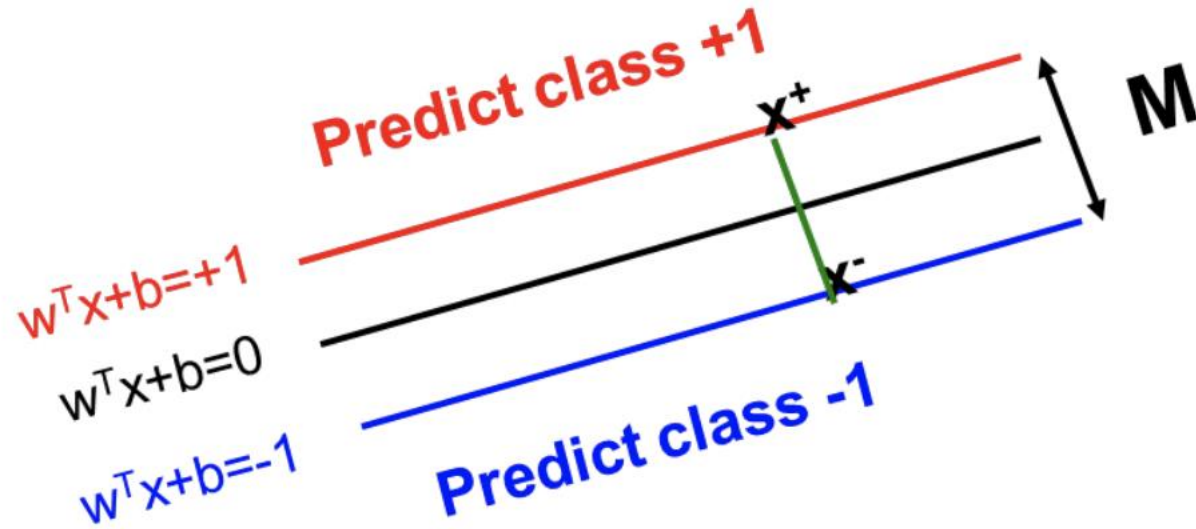
Classify as -1 if $w^T x + b \leq -1$

Undefined if $-1 < w^T x + b < 1$

Let u and v be two points on the +1 plane, then for the vector defined by u and v we have $w^T(u-v) = 0$

- Observation 1: the vector w is orthogonal to the +1 plane

Calculating M: Observation 2



Classify as +1 if $w^T x + b \geq 1$

Classify as -1 if $w^T x + b \leq -1$

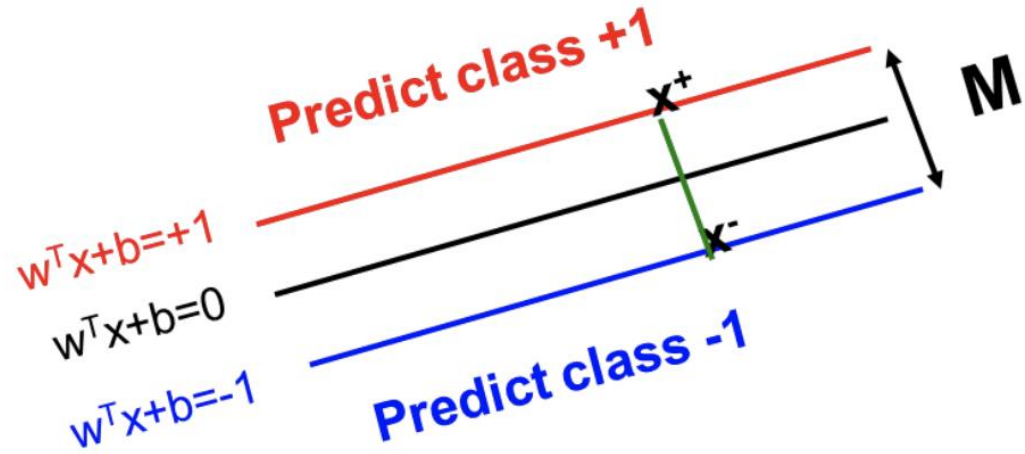
Undefined if $-1 < w^T x + b < 1$

- Observation 1: the vector w is orthogonal to the +1 and -1 planes
- Observation 2: if x^+ is a point on the +1 plane and x^- is the closest point to x^+ on the -1 plane then

$$x^+ = \lambda w + x^-$$

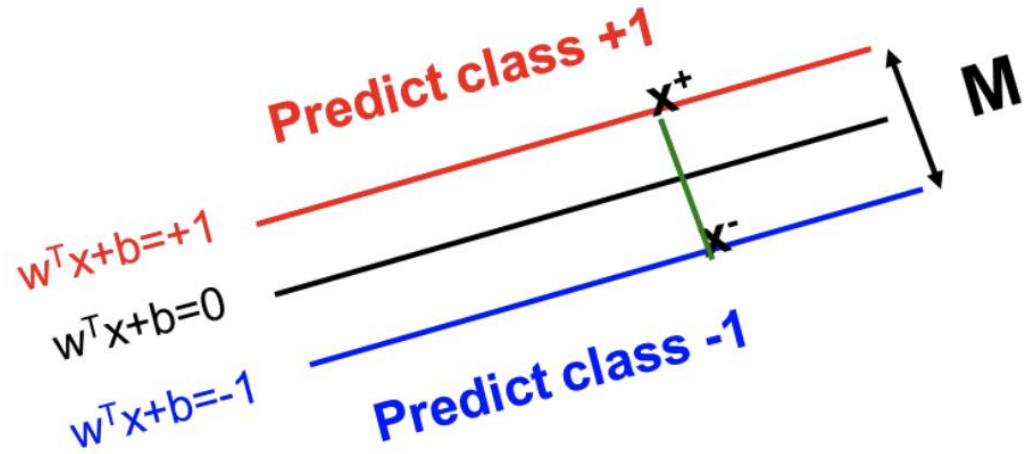
Since w is orthogonal to both planes we need to ‘travel’ some distance along w to get from x^+ to x^-

Calculating M: Putting it all together



- $w^T x^+ + b = +1$
- $w^T x^- + b = -1$
- $x^+ = \lambda w + x^-$
- $|x^+ - x^-| = M$

Calculating M: Putting it all together



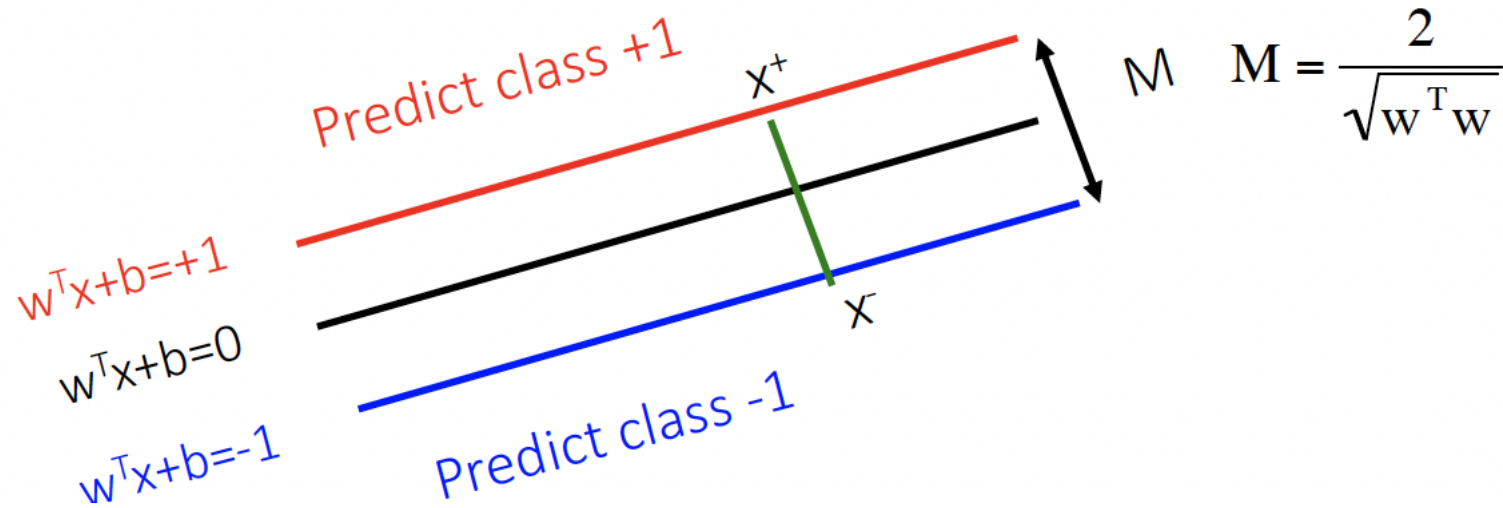
- $w^T x^+ + b = +1$
- $w^T x^- + b = -1$
- $x^+ = \lambda w + x^-$
- $|x^+ - x^-| = M$

Any questions?



SVM Optimization

Optimization Step: Learning Optimal Parameters for SVM

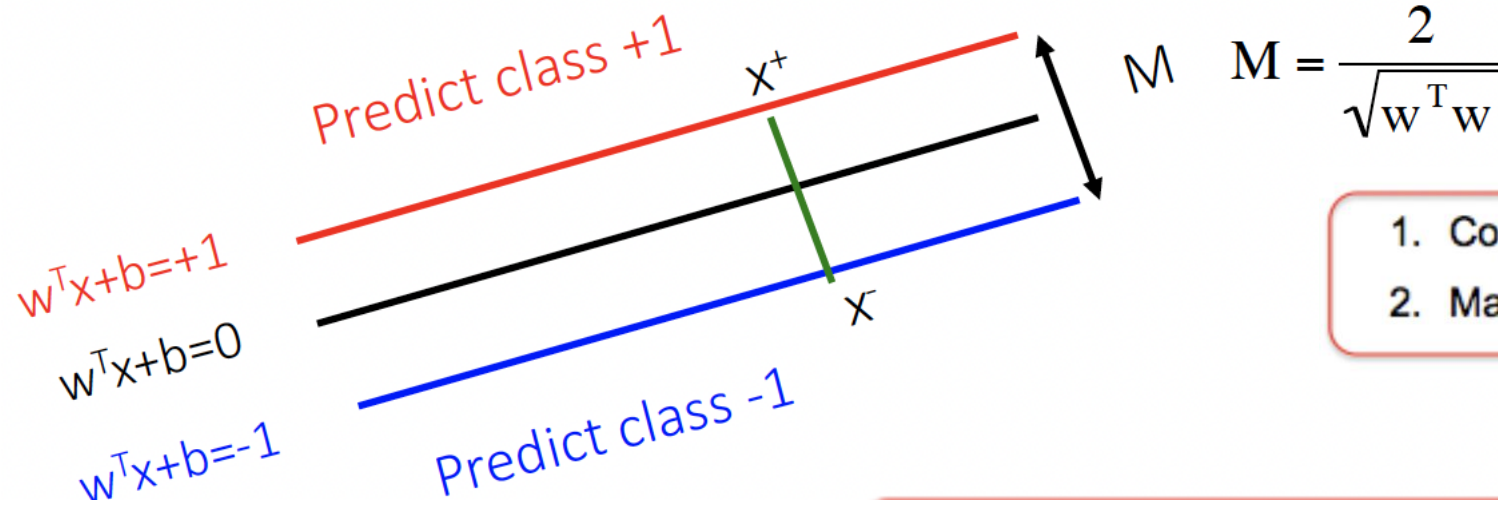


1. Correctly classifies all points
2. Maximizes the margin (or equivalently minimizes $w^T w$)

Min $(w^T w)/2$

subject to the following constraints:

Optimization Step: Learning Optimal Parameters for SVM



$$M = \frac{2}{\sqrt{w^T w}}$$

1. Correctly classifies all points
2. Maximizes the margin (or equivalently minimizes $w^T w$)

$$\text{Min } (w^T w)/2$$

subject to the following constraints:

For all x in class + 1

$$w^T x + b \geq 1$$

For all x in class - 1

$$w^T x + b \leq -1$$

A total of n constraints if we have n training samples

Reformulating Optimization Step

1. Correctly classifies all points
2. Maximizes the margin (or equivalently minimizes $w^T w$)

$$\text{Min } (w^T w)/2$$

subject to the following constraints:

For all x in class + 1

$$w^T x + b \geq 1$$

For all x in class - 1

$$w^T x + b \leq -1$$

Final SVM Optimization

1. Correctly classifies all points
2. Maximizes the margin (or equivalently minimizes $w^T w$)

$$\begin{aligned} \min_{w,b} \quad & \frac{1}{2} \|w\|^2 \\ \text{subject to} \quad & y_i(w^T x_i + b) \geq 1, \quad \forall i \end{aligned}$$

Why square the L2-norm?

- Convex
- Smooth
- Quadratic

Quadratic programming i.e.,

- Quadratic objective
- Linear constraints

Class activity

Support Vector Machine

Recap – Learn about kernels and support vector machine (SVM)

(1) Kernelized Perceptron

(2) Linear SVM

(3) Defining maximum margin

(4) SVM Optimization

(5) Class activity: SVM

Wrap up



What was the clearest point today?



menti.com
6320 1646

What was the muddiest point today?

